

УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
НОВИ САД
Департман за рачунарство и аутоматику
Одсек за рачунарску технику и рачунарске комуникације

ИСПИТНИ РАД

Кандидат: **Ивана Милинковић**
Број индекса: **РА 221/2023**

Предмет: **Основи паралелног програмирања и софтверски алати**
Тема рада: **MAVH преводилац**

Ментор рада: **др Миодраг Ђукић**

Нови Сад, јун, 2026.

Sadržaj

1. **Uvod**
   - 1.1 Problem MAVN prevodioca
   - 1.2 Zadatak
2. **Analiza problema**
3. **Koncept rešenja**
4. **Programsko rešenje**
   - 4.1 Main
   - 4.2 Constants
   - 4.3 Types
   - 4.4 Token
   - 4.5 IR
     - 4.5.1 Variable
     - 4.5.2 Instruction
   - 4.6 Finite State Machine
   - 4.7 Lexical Analysis
   - 4.8 Syntax Analysis
   - 4.9 Liveness Analysis
   - 4.10 Interference Graph
   - 4.11 Resource Allocation
5. **Verifikacija**
6. **Zaključak**

1. Uvod

1.1 Problem MAVN prevodioca

MAVN (Mips Asembler Visokog Nivoa) je alat koji prevodi program napisan na višem MIPS 32bit asemblerskom jeziku na osnovni asemblerski jezik. Viši MIPS 32bit asemblerski jezik služi lakšem asemblerskom programiranju jer uvodi koncept **registarske promenljive**. Registarske promenljive omogućavaju programerima da prilikom pisanja instrukcija koriste promenljive umesto pravih resursa. Ovo znatno olakšava programiranje jer programer ne mora da vodi računa o korišćenim registrima i njihovom sadržaju. Radi jednostavnosti projekta u instrukcijama ne smeju da se koriste pravi resursi već isključivo promenljive.

MAVN jezik podržava 10 osnovnih MIPS instrukcija, a to su:

- **add** – (addition) sabiranje
- **addi** – (addition immediate) sabiranje sa konstantom
- **b** – (unconditional branch) bezuslovni skok
- **bltz** – (branch on less than zero) skok ako je registar manji od nule
- **la** – (load address) učitavanje adrese u registar
- **li** – (load immediate) učitavanje konstante u registar
- **lw** – (load word) čitavanje jedne memorijske reči
- **nop** – (no operation) instrukcija bez operacije
- **sub** – (subtraction) oduzimanje
- **sw** – (store word) upis jedne memorijske reči

Pored navedenih 10 instrukcija, dodate su još tri instrukcije:

- **beq** – (branch on equal) uslovni skok ako su dva registra jednaka (nije iz aritmetičko-logičkog skupa)

- **mul** – (multiply) množenje dva registra
- **and** – (logical AND) logičko I nad dva registra

1.2 Zadatak

Zadatak MAVN prevodioca je da učitanu ulaznu datoteku pisanu na MAVN jeziku prevede na MIPS 32bit asembler. Ograničiti se na jednu ulaznu datoteku. Koristiti ekstenziju ".mavn" za ulaznu datoteku koja sadrži program na MAVN jeziku.

Prevodilac treba da prilikom prevođenja:

1. Dodeli resurse za registarske promenljive – ograničiti se na 4 registra: t0, t1, t2 i t3 iz MIPS arhitekture
2. Sve memorijske promenljive generiše u sekciju za podatke - .data vodeći računa o sintaksi asemblerskog jezika
3. Sve instrukcije smesti u programsku sekciju - .text
4. Ime funkcije generiše kao globalni simbol - .globl i kao labelu na prvu njenu instrukciju
5. Generiše izlaznu datoteku sa ekstenzijom ".s" koja sadrži preveden i korektan MIPS 32bit asemblerski jezik polaznog programa

Takođe prilikom prevođenja treba omogućiti detekciju i reagovanje na leksičke i sintaksne greške.

2. Analiza problema

Pridržavajući se sintakse MAVN jezika koja je opisana gramatikom ispod, neophodno je izvršiti sve operacije i generisati ispravnu datoteku sa ".s" ekstenzijom.

Sintaksa MAVN jezika opisana je gramatikom:

```
Q → S ; L        S → _mem mid num      L → eof        E → add rid, rid, rid
                 S → _reg rid          L → Q          E → addi rid, rid, num
                 S → _func id                         E → sub rid, rid, rid
                 S → id: E                            E → la rid, mid
                 S → E                                E → lw rid, num(rid)
```

```
E → li rid, num
E → sw rid, num(rid)
E → b id
E → bltz rid, id
E → nop
E → beq rid, rid, id
E → mul rid, rid, rid
E → and rid, rid, rid
```

Terminalni simboli MAVN jezika su: **: ; , ( )**

`_mem, _reg, _func, num, id, rid, mid, eof, add, addi, sub, la, lw, li, sw, b, bltz, nop, beq, mul, and`

Deklaracije:

- **Deklaracija funkcije:** _func funcName – funcName mora početi slovom, u nastavku može biti bilo koji niz slova i brojeva.
- **Deklaracija memorijske promenljive:** _mem varName value – varName mora početi malim slovom m, u nastavku može biti bilo koji broj.
- **Deklaracija registarske promenljive:** _reg varName – varName mora početi malim slovom r, u nastavku može biti bilo koji broj.

3. Koncept rešenja

Kako bismo realizovali MAVN prevodilac, neophodno je izvršiti sledeće korake:

1. **Učitavanje ulazne datoteke** tipa .mavn
2. **Leksička analiza** – tokenizacija ulaznog programa pomoću konačnog determinističkog automata (FSM)
3. **Sintaksna analiza** – parsiranje tokena prema gramatici MAVN jezika algoritmom rekurzivnog spuštanja
4. **Analiza životnog veka** – određivanje def/use skupova, izgradnja grafa toka kontrole (succ/pred) i iterativno računanje in/out skupova
5. **Formiranje grafa smetnji** – kreiranje matrice interferencije između registarskih promenljivih
6. **Alokacija resursa** – bojenje grafa sa 4 boje ($t0-$t3) pomoću algoritma simplifikacije i selekcije, sa spill mehanizmom
7. **Generisanje izlaznog fajla** – MIPS 32bit asemblerski kod sa ekstenzijom ".s"

Tok izvršavanja:

```
.mavn → Učitavanje → Leksička analiza → Sintaksna analiza → Analiza životnog veka → Graf smetnji → Alokacija resursa → Generisanje koda → .s
```

4. Programsko rešenje

4.1 Main

Predstavlja glavni modul MAVN prevodioca. U ovom modulu se redom izvršavaju sve faze prevođenja: leksička analiza, sintaksna analiza, analiza životnog veka, formiranje grafa smetnji i alokacija resursa. Takođe sadrži funkciju za generisanje izlaznog MIPS koda u datoteku sa ekstenzijom ".s".

Pomoćne funkcije u main modulu:

- string getRegName(Variable* var) – vraća naziv fizičkog registra ($t0-$t3) dodeljenog promenljivoj
- void loadSpilled(ofstream& outFile, Variable* var) – generiše lw instrukciju za učitavanje spilled promenljive iz memorije
- void storeSpilled(ofstream& outFile, Variable* var) – generiše sw instrukciju za čuvanje spilled promenljive u memoriju
- void generateMipsCode(...) – generiše kompletan MIPS asemblerski kod u izlaznu datoteku

4.2 Constants

U ovom fajlu nalaze se konstante korišćene tokom formiranja MAVN prevodioca:

- IDLE_STATE = 0 – stanje mirovanja konačnog automata
- START_STATE = 1 – startno stanje konačnog automata
- INVALID_STATE = -2 – nevalidno stanje konačnog automata
- NUM_STATES = 51 – broj mogućih stanja (prvobitno 45, prošireno za instrukcije beq, mul, and)
- NUM_OF_CHARACTERS = 47 – broj dozvoljenih karaktera

- __INTERFERENCE__ = 1 – označava smetnju između dva registra
- __EMPTY__ = 0 – označava odsustvo smetnje
- __REG_NUMBER__ = 4 – broj registara koji se mogu koristiti

4.3 Types

Modul koji sadrži tipove tokena, instrukcija i registara. Predstavljeni su pomoću enuma.

- **enum TokenType** – sadrži sve tipove tokena
- **enum InstructionType** – sadrži sve tipove instrukcija
- **enum Regs** – sadrži registre: no_assign, t0, t1, t2, t3

4.4 Token

Modul koji se koristi za rad sa tokenima. Sadrži klasu Token.

Atributi:

- TokenType tokenType – tip tokena
- string value – vrednost tokena

Funkcije:

- TokenType getType() – vraća tip tokena
- void setType(TokenType t) – postavlja tip tokena
- string getValue() – vraća vrednost tokena
- void setValue(string s) – postavlja vrednost tokena
- void makeToken(int begin, int end, vector<char>& program, int lastFiniteState) – kreira token
- void makeErrorToken(int pos, vector<char>& program) – kreira token greške
- void makeEofToken() – kreira token kraja fajla
- void printTokenInfo() – ispisuje informacije o tokenu
- void printTokenValue() – ispisuje vrednost tokena

4.5 IR

4.5.1 Variable

Atributi:

- VariableType m_type – tip promenljive (MEM_VAR, REG_VAR, NO_TYPE)
- string m_name – ime promenljive
- int m_position – pozicija promenljive
- Regs m_assignment – dodeljeni registar
- int m_value – vrednost promenljive (za memorijske promenljive)
- bool m_spilled – oznaka da li je promenljiva sačuvana u memoriji umesto registra

Funkcije:

- string getName() / void setName(string)
- int getPosition() / void setPosition(int)
- VariableType getType() / void setType(VariableType)
- Regs getAssignment() / void setAssignment(Regs)
- int getValue() / void setValue(int)
- bool isSpilled() / void setSpilled(bool)

4.5.2 Instruction

Atributi:

- int m_position – pozicija instrukcije
- InstructionType m_type – tip instrukcije
- Variables m_dst – lista destination promenljivih
- Variables m_src – lista source promenljivih
- Variables m_use – lista korišćenih promenljivih
- Variables m_def – lista definisanih promenljivih
- Variables m_in – skup ulaznih živih promenljivih
- Variables m_out – skup izlaznih živih promenljivih
- list<Instruction*> m_succ – lista naslednika
- list<Instruction*> m_pred – lista prethodnika
- string m_label – labela instrukcije
- string m_targetLabel – ciljna labela za instrukcije skoka
- int m_numConst – numerička konstanta

Funkcije:

Getteri i setteri za sve atribute, uključujući getDst(), getSrc(), getUse(), getDef(), getIn(), getOut(), getSucc(), getPred().

4.6 Finite State Machine

Modul koji implementira konačni deterministički automat za leksičku analizu. Automat koristi matricu prelaza sa 51 stanjem i 47 podržanih karaktera. Dodata su 6 novih stanja (45-50) za prepoznavanje ključnih reči beq, mul i and:

- Stanje 45: prepoznaje "be" (međustanje ka beq)
- Stanje 46: prepoznaje "beq" (T_BEQ)
- Stanje 47: prepoznaje "mu" (međustanje ka mul)
- Stanje 48: prepoznaje "mul" (T_MUL)
- Stanje 49: prepoznaje "an" (međustanje ka and)
- Stanje 50: prepoznaje "and" (T_AND)

4.7 Lexical Analysis

Leksički analizator se realizuje pomoću konačnog determinističkog automata čiji je cilj da prepozna sve simbole.

Atributi:

- ifstream inputFile – ulazna datoteka
- vector<char> programBuffer – programski bafer
- unsigned int programBufferPosition – pozicija u baferu
- FiniteStateMachine fsm – konačni automat
- TokenList tokenList – lista tokena
- Token errorToken – token greške

Funkcije:

- void initialize() – inicijalizacija leksičkog analizatora i FSM
- bool Do() – izvršava leksičku analizu
- bool readInputFile(string fileName) – učitava ulaznu datoteku
- Token getNextTokenLex() – vraća sledeći token
- TokenList& getTokenList() – vraća listu tokena
- void printTokens() – ispisuje tokene
- void printLexError() – ispisuje grešku

4.8 Syntax Analysis

Sintaksni analizator implementiran algoritmom rekurzivnog spuštanja. Parsira listu tokena prema gramatici MAVN jezika i gradi interni prikaz programa.

Atributi:

- LexicalAnalysis& lexicalAnalysis – referenca na leksički analizator
- TokenList::iterator currentToken – iterator kroz listu tokena
- Instructions instructions – lista instrukcija
- Variables variables – lista promenljivih
- map<string, int> labels – mapa labela
- string functionName – ime funkcije
- int instructionPosition – brojač instrukcija

Funkcije:

- bool Do() – izvršava sintaksnu analizu
- void Q(), S(), L(), E() – neterminalne funkcije prema gramatici
- void eatToken(TokenType) – konzumira očekivani token ili prijavljuje grešku
- Variable* findVariable(string) – pronalazi promenljivu po imenu
- void printSyntaxError(Token) – ispisuje sintaksnu grešku
- Instructions& getInstructions() – vraća listu instrukcija
- Variables& getVariables() – vraća listu promenljivih
- string getFunctionName() – vraća ime funkcije
- map<string, int>& getLabels() – vraća mapu labela

4.9 Liveness Analysis

Analiza životnog veka određuje za svaku instrukciju koje su promenljive žive (sadrže vrednost koja će biti korišćena u budućnosti). Da bi se omogućilo da različite promenljive koje nisu istovremeno u upotrebi dele isti resurs, neophodna je informacija o životnom veku promenljive.

Atributi:

- Instructions& instructions – referenca na listu instrukcija
- map<string, int>& labels – referenca na mapu labela

Funkcije:

- void Do() – izvršava kompletnu analizu životnog veka
- void buildControlFlowGraph() – gradi graf toka kontrole (succ/pred veze)
- void setDefUse() – postavlja def i use skupove za svaku instrukciju
- void computeLiveness() – iterativni algoritam za računanje in/out skupova
- void printLivenessInfo(int dumps) – ispisuje rezultate analize

4.10 Interference Graph

Graf smetnji prikazuje koje registarske promenljive ne mogu deliti isti fizički registar jer su istovremeno žive. Predstavljen je matricom smetnji gde kolone i redovi predstavljaju registarske promenljive, a na preseku se nalazi indikator smetnje.

Atributi:

- Instructions& instructions – referenca na listu instrukcija
- Variables& variables – referenca na listu promenljivih
- Variables regVariables – lista registarskih promenljivih
- vector<vector<int>> matrix – matrica smetnji

Funkcije:

- void build() – gradi matricu smetnji
- vector<vector<int>>& getMatrix() – vraća matricu smetnji
- Variables& getRegVariables() – vraća listu registarskih promenljivih
- void printInterferenceMatrix() – ispisuje matricu smetnji

4.11 Resource Allocation

Faza dodele resursa ima zadatak da neograničeni broj registarskih promenljivih svede na tačno 4 registra ($t0-$t3) koliko ima na ciljnoj platformi. Koristi algoritam bojenja grafa.

Atributi:

- InterferenceGraph& interferenceGraph – referenca na graf smetnji
- stack<Variable*> simplifyStack – stek za fazu simplifikacije

Funkcije:

- bool Do() – izvršava alokaciju resursa (simplifikacija + selekcija)
- bool simplify() – uklanja čvorove sa stepenom < 4 i stavlja ih na stek. Ako nijedan čvor nema stepen < 4, bira se čvor sa najvećim stepenom kao potencijalni spill kandidat.
- bool select() – skida čvorove sa steka i dodeljuje registre. Ako nijedan registar nije slobodan, promenljiva se označava kao spilled.
- int getDegree(int, vector<vector<int>>&) – vraća stepen čvora
- void printAllocation() – ispisuje rezultate alokacije

5. Verifikacija

5.1 Provere

Da bi program mogao da funkcioniše bez grešaka, uvedene su sledeće provere:

5.1.1 Funkcije i labele

- Funkcije i labele moraju početi slovom, nakon toga može da se nalazi bilo koji niz alfanumeričkih znakova
- Korišćenje nedefinisane funkcije ili labele rezultuje sintaksnom greškom

5.1.2 Memorijska promenljiva

- Memorijska promenljiva mora početi malim slovom m, nakon toga može da se nalazi bilo koji broj
- Korišćenje nedefinisane memorijske promenljive rezultuje greškom

5.1.3 Registarska promenljiva

- Registarska promenljiva mora početi malim slovom r, nakon toga može da se nalazi bilo koji broj
- Korišćenje nedefinisane registarske promenljive rezultuje greškom

5.1.4 Alokacija resursa i spill

- Ukoliko broj istovremeno živih promenljivih premašuje broj dostupnih registara (4), promenljiva se čuva u memoriji (spill mehanizam)
- Za spilled promenljive se u .data sekciji generiše lokacija za čuvanje
- Pre svake upotrebe spilled promenljive generiše se lw instrukcija, a posle svakog upisa sw instrukcija

5.2 Rezultati

Program je uspešno izgenerisao izlazne datoteke simple.s i multiply.s za priložene datoteke simple.mavn i multiply.mavn.

simple.mavn → simple.s

Ulaz (simple.mavn):

```
_mem m1 6;
_mem m2 5;
_reg r1;
_reg r2;
_reg r3;
_reg r4;
_reg r5;
_func main;
    la      r4,m1;
    lw      r1, 0(r4);
    la      r5, m2;
    lw      r2, 0(r5);
    add     r3, r1, r2;
```

Izlaz (simple.s):

```
.globl main

.data
m1:    .word 6
m2:    .word 5

.text
main:
    la      $t0, m1
    lw      $t1, 0($t0)
    la      $t0, m2
    lw      $t0, 0($t0)
    add     $t0, $t1, $t0
```

multiply.mavn → multiply.s

Ulaz (multiply.mavn):

```
_mem m1 6;
_mem m2 5;
_mem m3 0;
_reg r1;  _reg r2;  _reg r3;  _reg r4;
_reg r5;  _reg r6;  _reg r7;  _reg r8;
_func main;
    la      r1, m1;
    lw      r2, 0(r1);
    la      r3, m2;
    lw      r4, 0(r3);
    li      r5, 1;
```

```
    li      r6, 0;
lab:
    add     r6, r6, r2;
    sub     r7, r5, r4;
    addi    r5, r5, 1;
    bltz    r7, lab;
    la      r8, m3;
    sw      r6, 0(r8);
    nop;
```

Izlaz (multiply.s):

```
.globl main

.data
m1:             .word 6
m2:             .word 5
m3:             .word 0
__spill_r2:     .word 0

.text
main:
    la      $t0, m1
    lw      $t0, 0($t0)
    sw      $t0, __spill_r2
    la      $t1, m2
    lw      $t3, 0($t1)
    li      $t2, 1
    li      $t1, 0
lab:
    lw      $t0, __spill_r2
    add     $t1, $t1, $t0
    sub     $t0, $t2, $t3
    addi    $t2, $t2, 1
    bltz    $t0, lab
    la      $t0, m3
    sw      $t1, 0($t0)
    nop
```

Kod primera multiply.mavn javlja se situacija da je 5 registarskih promenljivih istovremeno živo u petlji, a na raspolaganju su samo 4 registra. Problem je rešen spill mehanizmom – promenljiva r2 se čuva u memorijskoj lokaciji __spill_r2 i učitava pre svake upotrebe u petlji.

6. Zaključak

Program podržava detekciju leksičkih i sintaksnih grešaka kao i generisanje odgovarajućih izveštaja o eventualnim greškama. Implementirane su sve faze MAVN prevodioca: leksička analiza, sintaksna analiza, analiza životnog veka, formiranje grafa smetnji, alokacija resursa i generisanje izlaznog koda. Pored 10 osnovnih MAVN instrukcija, dodate su tri dodatne instrukcije: beq (uslovni skok – nije iz aritmetičko-logičkog skupa), mul (množenje) i and (logičko I). Alokacija resursa je ograničena na 4 registra ($t0-$t3) iz MIPS arhitekture, uz implementiran spill mehanizam za slučaj kada broj istovremeno živih promenljivih prevazilazi broj dostupnih registara. Nakon uspešno implementiranih svih faza, ulazni program je preveden u korektan MIPS 32bit asemblerski kod sa ekstenzijom ".s".